

Assignment 5

Step 1: Problem Statement

Create a two part project. In Part 1 we'll create a JCF stack and populate it from an Integer array. We'll replace all zeros with 0's then print all values on the stack while preserving its original state.

In Part 2 - we'll create a Generic stack method and from that create 2 unsorted stacks populated by reading 2 files. We will then sort and merge the 2 previously unsorted stacks. This must be done in a way that works for any 2 Integer stacks or String stacks.

Step 2: What I know

I understand the concept of stacks and the methods associated with them.

I understand how to create concrete classes and methods

I understand how to read from files to populate the stacks.

Step 2: what I don't know

Need work on generic classes
+ ~~membership~~ Methods

Need work on sort + Merge
Methods.

Step 3: Pseudo code main + sort stack

Part 2 create File objects

New File1
New File2
New File3
New File4

Create Scanners for each file

Scanner 1

// 2

// 3

// 4

* Read files to populate stacks

while (file not empty) {

stack.push(next());

}

// Repeat for each stack

create array new

int[] numbers = {43, 99, 1, 99};

for (int i = 0; i < numbers.length; i++) {

```
stack.push (members[i]);  
}
```

Call replaceZerosWithTen()

Call printstack(stack)

// Create Gen Stack

```
GenericStack<Integer> numStack = new GenStack  
    "  
    "<String> StringStack = new GenStack
```

Read files to populate Int

```
Stacks  
while (File not empty) {  
    numStack.push (nextInt());  
}
```

Read files to populate String
Stack

```
while (File not empty) {  
    StringStack.push (next());  
}
```

Call Gen PrintStack();

Call ~~Gen~~ SortStack();

Call MergeStacks();

Call Gen PrintStack();

~~SortStack~~

```
public static <E extends Comparable<E>> void sortStack  
(GenericStack<E> aStack)
```

Step 3 (cont)
~~For~~ ~~Stack~~ Compare to (a stack)

}
loop to compare & replace each
value

~~{ if > stack, compare to stack }~~

Step 4: What I learned

Sorting stacks with limited resources
is a challenge and not very
efficient.

Generic methods/classes very
handy for giving flexibility of
data types. Allowed me to
write code once and use with
different data types.

Could have made the code much
more efficient with a little
extra time but was trying to
get a clear, if not concise,
working code I could submit